



COURSE CODE: CSC421

COURSE TITLE: Database Management II

EXAMINATION: B.Sc. Examination

PRACTICE QUESTIONS

Question 1

Case Study: Consider a university database with tables: Department (DeptName, Building, Budget), Instructor (ID, Name, Salary, DeptName), Student (ID, Name, DOB, DeptName), Course (CourseCode, Title, Credits, DeptName), and Takes (CourseCode, SectionID, Semester, Year, StudentID, Grade). The university wants to automate salary calculations and eligibility checks.

- (a) Write a **PostgreSQL function** that takes a **department name** as input and **returns the average salary** of instructors in that department. **[5 marks]**
- (b) Modify the function from part (a) to **return a table** with instructor IDs, names, and salaries for all instructors in the given department whose salary is above the average. **[5 marks]**
- (c) Explain how this function could be used in a query to filter departments where the average salary exceeds 100,000, and provide an example query. **[5 marks]**

Question 2

Case Study: In an e-commerce platform like the one in Lecture 2 and the project, products have varying attributes (e.g., colour variants with quantities and prices). The **Products** table uses a **JSONB** column for product_info (e.g., [{"colour": "Red", "quantity": 10, "price": 50.00}, {"colour": "Blue", "quantity": 5, "price": 55.00}]) and a text array for tags.

- (a) Write a **PostgreSQL query** to find all products that have **at least one** variant with "colour": "Black" and quantity greater than 20. **[4 marks]**
- (b) Write an **UPDATE** statement to increase the price of the first variant **by 10%** for a specific product (assume product_id = 1). **[5 marks]**
- (c) Write a query to list the **product name** and the **price of the second variant** (if it exists) for products with at least two variants. Explain the difference between **#>** and **#>>** in this context. **[6 marks]**

Question 3

Based on Lecture 1's discussion of transactions and indexing in a university setup with tables like Instructor (ID, Name, DeptName, Salary) and Department (DeptName, Building, Budget).

- (a) Explain the **ACID** properties of transactions with one example from a database operation (e.g., updating instructor salaries). **[4 marks]**
- (b) Write a **PostgreSQL transaction block** that increases the budget of a department by the total salary of its instructors, ensuring atomicity if any error occurs. **[5 marks]**

- (c) Create a **B+ Tree** index on the **Name** column of the **Instructor** table with the appropriate **SQL command**. Explain when a Hash Index might be more appropriate than a B+ Tree Index, and provide an example scenario from the university database. [6 marks]

Question 4

Case Study: Extending the university database from Lecture 3, the administration wants to enforce rules automatically. For instance, when a new instructor joins, the department budget should adjust, and student eligibility for courses based on prerequisites should be checked.

- (a) Write a PostgreSQL procedure that **takes an instructor ID** and **new salary** as input, **updates the instructor's salary**, and then **adjusts the department's budget** by adding the salary difference (handle cases where the department has no instructors). [6 marks]
- (b) Create a trigger and its accompanying function that fires **AFTER an INSERT** on the **Instructor** table to log the new instructor's details into a separate audit table (assume an **Audit** table exists with columns: **InstructorID**, **Timestamp**). [6 marks]
- (c) Explain the difference between **BEFORE** and **AFTER** triggers. [3 marks]

Question 5

Case Study: Consider a MongoDB database for a university with collections: **Student** (documents with fields like **_id**, **name**, **dept_name**, **courses**: array of course objects), **Instructor**, and **Course**. Documents are flexible, e.g., a **Student** document: { "_id": 1, "name": "Alice", "dept_name": "CS", "courses": [{"code": "CSC421", "grade": "A"}, {"code": "MATH101", "grade": "B"}] }.

- (a) Write a MongoDB query to find all students in the **CS** department, displaying only their names and department names. [4 marks]
- (b) Insert a new student document with an **embedded array for courses**, then update it to add an email field. [5 marks]
- (c) Explain how this document-based approach in MongoDB resolves challenges of semi-structured data (e.g., varying course counts per student) compared to a relational schema in PostgreSQL. Give **one** advantage and **one** disadvantage. [6 marks]

Question 6

Case Study: Consider an e-commerce database in PostgreSQL, with tables: **Manufacturers** (**id**, **manufacturer**), **Products** (**product_id**, **pname**, **description**, **brand**, **product_info**: **JSONB**, **tags**: **text[]**, **price**), **Users** (**user_id**, **user_details**: **person**, **favourites**: **text[]**), and **Orders** (**orderid**, **customer_id**, **orderdetails**: **JSONB[]**, **total_amount**). The type **person** is a user-defined type with the following components: **address**, **phone**, and **email** (all of the type **TEXT**).

- (a) Write a PostgreSQL procedure to place an order: It takes **customer_id** and a **JSONB array of order details** (e.g., [{"product_id": 1, "quantity": 2}]), calculates **total_amount**, generates **orderid** (**timestamp** + **user_id**), and inserts the order. [Hint: A loop and **jsonb_array_elements** function are handy tools]. [7 marks]
- (b) Using **array operators**, write a query to find users whose favourites array contains **both 'laptop' and 'headphones'**. [4 marks]

- (c) Explain how using JSONB for orderdetails and product_info handles diverse product variants, and provide an example of using the deletion operator (-) to remove a specific variant from product_info. **[4 marks]**